

Visual Product Search Engine

YOLO-Assisted Cropping + CLIP Retrieval with Caption Fusion

Course: Visual Recognition

Dataset: DeepFashion In-Shop Clothes Retrieval

Team Members (Roll Nos.):

Pasupuleti T A N S V Vignesh (IMT2023003)

Kunda Sri Krishna Chaitanya (IMT2023039)

Bojja Sunhith Reddy (IMT2023113)

Jagini Mohit (IMT2023528)

GitHub Repository:

https://github.com/codetuscan/vr_final_project_app.git

Inference Script Path:

`vr_final_project_app/inference/kaggle_batch_eval.py`

Date: May 15, 2026

This report documents the end-to-end pipeline, ablation study, and Streamlit demo application.

Contents

Abstract	3
1 Problem Statement	4
2 Dataset	5
2.1 Splits and Files	5
2.2 Annotations	5
3 Methodology	6
3.1 Overall Retrieval Pipeline	6
3.2 Detection (YOLO) Fine-tuning	6
3.3 Embedding and Fusion	7
3.3.1 CLIP Image Embedding	7
3.3.2 Caption Fusion (Condition B/C)	7
3.3.3 Fast Retrieval with HNSW	7
3.4 Metrics	7
4 Ablation Study	8
4.1 Condition A: Vision-Only CLIP (Baseline)	8
4.2 Condition B: Frozen CLIP + Frozen BLIP-2 (Caption Fusion)	8
4.3 Condition C: Fine-tuned CLIP + Frozen BLIP-2	9
5 Results and Trends	11
5.1 Aggregated Results (from Experiment Outputs)	11
5.2 Within-Ablation Comparisons	11
5.3 Across-Ablation Comparisons	12
6 Streamlit Application	14
6.1 User Flow	14
6.2 Application Screenshots	15

6.3	Software Architecture	16
6.4	Configuration and Execution	16
7	Key Findings and Takeaways	18
8	Conclusion	19

Abstract

We build an end-to-end **visual product search engine** for fashion items in the DeepFashion In-Shop setting. Given a query image, the system localizes the clothing region using a fine-tuned **YOLO** detector, embeds the crop using **CLIP**, and retrieves similar items from a gallery using a cosine-similarity **HNSW** index. To incorporate semantic cues, we optionally fuse image embeddings with caption-derived text embeddings generated by **BLIP-2**, and apply an optional **BLIP-ITM** reranker to refine the top- K results.

Our best-performing configuration is **Condition C** (fine-tuned CLIP + caption fusion) with $\alpha = 0.7$, which achieves strong retrieval quality across all K values (e.g., $\text{HR@15} = 0.9374$, $\text{Recall@15} = 0.6614$, $\text{mAP@15} = 0.3564$, $\text{NDCG@15} = 0.5015$). Overall, the results show that **fine-tuning the visual embedding space is the primary driver of improvement**, while fusion helps most when the visual signal is weighted more heavily.

1. Problem Statement

The goal of this project is to build a **Visual Product Search Engine** for fashion items. Given a **query image** captured in the wild (or a catalog-style image), the system should retrieve visually and semantically similar products from a **gallery** of known items.

The key challenges are:

- **Localization:** query images may contain background clutter; the clothing region must be isolated.
- **Viewpoint and appearance changes:** the same item can appear in different poses (front/back/side), lighting, or partial occlusions.
- **Semantic similarity:** purely visual features may miss category/style cues (e.g., “floral dress” vs. “solid dress”).

We address these by combining:

- a fine-tuned **YOLO** detector to crop the clothing region,
- **CLIP** embeddings for robust image representations,
- optional **caption fusion** using **BLIP-2** captions and CLIP text embeddings,
- fast approximate nearest neighbor search using **HNSW**.

2. Dataset

We follow the standard **in-shop clothes retrieval** setup used by the DeepFashion In-Shop dataset format. The dataset is organized around an **item identity** (`item_id`). Multiple images correspond to the same item identity, captured from different viewpoints.

2.1 Splits and Files

The dataset provides an evaluation partition file (`list_eval_partition.txt`) that assigns each image to one of:

- **train:** used for detector training and (in Condition C) representation learning,
- **gallery:** reference set used to build the retrieval index,
- **query:** used for evaluation by retrieving relevant gallery items.

In our experimental setup (as produced by reading the partition file in the ablation notebooks), the evaluation split contained **12,612 gallery** images and **14,218 query** images.

2.2 Annotations

For detection training, the dataset includes bounding-box annotations (`list_bbox_inshop.txt`). Each annotation contains pixel-space coordinates (x_1, y_1, x_2, y_2) for the clothing region. These boxes are used to create a YOLO-format dataset.

3. Methodology

3.1 Overall Retrieval Pipeline

For all ablation conditions, we use the same retrieval backbone:

1. **YOLO crop:** run the detector on each image and select the largest bounding box; apply padding and crop.
2. **Embedding:** compute CLIP embeddings on the cropped image (and optionally text embeddings from captions).
3. **Index:** build a cosine-similarity HNSW index over the gallery embeddings.
4. **Retrieve:** for each query, search the index for the top- K nearest gallery items.
5. **Evaluate:** compute HR@K, Recall@K, mAP@K, and NDCG@K for $K \in \{5, 10, 15\}$.

3.2 Detection (YOLO) Fine-tuning

We fine-tuned a YOLO detector on the training split. The training notebook constructs a YOLO dataset as follows:

- Load bounding boxes from `list_bbox_inshop.txt`.
- Join with the evaluation partition file and keep only **train** images.
- Convert each bounding box from pixel coordinates to YOLO format:

$$x_c = \frac{x_1 + x_2}{2W}, \quad y_c = \frac{y_1 + y_2}{2H}, \quad w = \frac{x_2 - x_1}{W}, \quad h = \frac{y_2 - y_1}{H}$$

where W, H are image width and height.

- Split into train/val (90/10) with stratification by `clothes_type`.
- Train using Ultralytics YOLO (YOLOv8 family), saving `best.pt`.

In the retrieval experiments, the trained detector is used **only for cropping** (no further YOLO training is performed).

3.3 Embedding and Fusion

3.3.1 CLIP Image Embedding

We use OpenCLIP (ViT-B-32, pretrained on OpenAI weights). For an image crop I , the image encoder outputs an embedding $\mathbf{v} = f_{\text{img}}(I)$. We apply L2 normalization:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}.$$

3.3.2 Caption Fusion (Condition B/C)

For a crop I , BLIP-2 generates a caption c . We embed it using the CLIP text encoder to obtain $\mathbf{t} = f_{\text{txt}}(c)$ and L2 normalize it.

We fuse image and text embeddings using a weighted sum and re-normalize:

$$\mathbf{z} = \alpha \hat{\mathbf{v}} + (1 - \alpha) \hat{\mathbf{t}}, \quad \hat{\mathbf{z}} = \frac{\mathbf{z}}{\|\mathbf{z}\|_2}.$$

We evaluate multiple α values to understand how visual vs. textual signals affect retrieval.

3.3.3 Fast Retrieval with HNSW

We use `hnswlib` with cosine distance. Key parameters used in experiments:

- $M = 32$, `efConstruction=200`, `efSearch=50`.

3.4 Metrics

For each query we assume relevance is defined by matching `item_id`.

- **Hit Rate@K (HR@K)**: 1 if at least one relevant item appears in top- K .
- **Recall@K**: fraction of all relevant gallery items retrieved in top- K .
- **mAP@K**: mean average precision up to rank K .
- **NDCG@K**: normalized discounted cumulative gain up to rank K .

To estimate variance, the evaluation is repeated across four seeds by shuffling the query order (seeds: 39, 3, 113, 528).

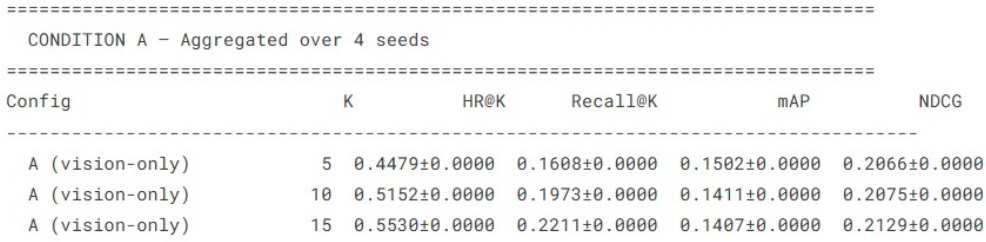
4. Ablation Study

We designed three experimental conditions to isolate the contribution of each component.

4.1 Condition A: Vision-Only CLIP (Baseline)

Goal: establish a baseline using only CLIP image embeddings.

Pipeline: YOLO crop $\rightarrow$ CLIP image embedding $\rightarrow$ HNSW retrieval. This corresponds to $\alpha = 1$ (no captions).



=====					
CONDITION A - Aggregated over 4 seeds					
=====					
Config	K	HR@K	Recall@K	mAP	NDCG

A (vision-only)	5	0.4479±0.0000	0.1608±0.0000	0.1502±0.0000	0.2066±0.0000
A (vision-only)	10	0.5152±0.0000	0.1973±0.0000	0.1411±0.0000	0.2075±0.0000
A (vision-only)	15	0.5530±0.0000	0.2211±0.0000	0.1407±0.0000	0.2129±0.0000

Figure 4.1: Condition A: aggregated metrics over 4 seeds (screenshot).

Ablation-specific observations.

- Condition A establishes the reference performance of vision-only retrieval.
- As K increases, HR@K and Recall@K increase, which is expected for nearest-neighbor retrieval.

4.2 Condition B: Frozen CLIP + Frozen BLIP-2 (Caption Fusion)

Goal: improve retrieval by adding semantic cues from captions.

Pipeline: YOLO crop $\rightarrow$ BLIP-2 caption $\rightarrow$ CLIP image/text embeddings $\rightarrow$ fusion with α $\rightarrow$ HNSW retrieval.

Additionally, we apply an optional **BLIP ITM** (image-text matching) reranker on the candidate list, using query crop + candidate captions.

Ablation-specific observations.

- Within Condition B, increasing α from 0.3 to 0.7 improves all reported metrics for all K values (e.g., HR@15: 0.4617 $\rightarrow$ 0.5655; NDCG@15: 0.1532 $\rightarrow$ 0.1968).

```

=====
CONDITION B - Aggregated over 4 seeds
=====

```

Config		K	HR@K	Recall@K	mAP	NDCG
B $\alpha=0.3$	5	0.3376 $\pm$ 0.0002	0.1211 $\pm$ 0.0000	0.0964 $\pm$ 0.0001	0.1384 $\pm$ 0.0001	
B $\alpha=0.3$	10	0.4229 $\pm$ 0.0001	0.1598 $\pm$ 0.0001	0.0951 $\pm$ 0.0001	0.1474 $\pm$ 0.0001	
B $\alpha=0.3$	15	0.4617 $\pm$ 0.0002	0.1797 $\pm$ 0.0001	0.0958 $\pm$ 0.0001	0.1532 $\pm$ 0.0001	

Figure 4.2: Condition B ($\alpha = 0.3$): aggregated metrics over 4 seeds (screenshot).

```

=====
CONDITION B - Aggregated over 4 seeds
=====

```

Config		K	HR@K	Recall@K	mAP	NDCG
B $\alpha=0.7$	5	0.4107 $\pm$ 0.0002	0.1527 $\pm$ 0.0001	0.1220 $\pm$ 0.0001	0.1734 $\pm$ 0.0001	
B $\alpha=0.7$	10	0.5142 $\pm$ 0.0001	0.2041 $\pm$ 0.0001	0.1216 $\pm$ 0.0001	0.1870 $\pm$ 0.0001	
B $\alpha=0.7$	15	0.5655 $\pm$ 0.0001	0.2335 $\pm$ 0.0001	0.1238 $\pm$ 0.0001	0.1968 $\pm$ 0.0001	

Figure 4.3: Condition B ($\alpha = 0.7$): aggregated metrics over 4 seeds (screenshot).

- This suggests that, with frozen captioning/text features, a higher visual weight is more robust for the dataset.

4.3 Condition C: Fine-tuned CLIP + Frozen BLIP-2

Goal: adapt CLIP to the dataset distribution while retaining the caption-fusion benefits.

Fine-tuning: We fine-tune the CLIP vision encoder on YOLO-cropped training images using an InfoNCE objective with positive pairs defined by matching `item_id`. To avoid overfitting and reduce compute, only the last N transformer blocks are unfrozen (`unfreeze_last_n=4`).

Retrieval: After fine-tuning, we compute gallery and query image embeddings, fuse with caption embeddings as in Condition B, build an HNSW index, and apply the same metric suite.

```

=====
CONDITION C - Aggregated
=====

```

Config		K	HR@K	Recall@K	mAP	NDCG
C $\alpha=0.3$	5	0.6817 $\pm$ 0.0004	0.3219 $\pm$ 0.0002	0.2465 $\pm$ 0.0001	0.3306 $\pm$ 0.0002	
C $\alpha=0.3$	10	0.8292 $\pm$ 0.0005	0.4682 $\pm$ 0.0004	0.2651 $\pm$ 0.0002	0.3800 $\pm$ 0.0003	
C $\alpha=0.3$	15	0.8925 $\pm$ 0.0003	0.5721 $\pm$ 0.0005	0.2812 $\pm$ 0.0002	0.4169 $\pm$ 0.0003	

Figure 4.4: Condition C ($\alpha = 0.3$): aggregated metrics (screenshot).

Ablation-specific observations.

```

=====
CONDITION C - Aggregated
=====
Config          K      HR@K      Recall@K      mAP      NDCG
-----
C alpha=0.7      5  0.7812±0.0000  0.3967±0.0001  0.3162±0.0000  0.4108±0.0000
C alpha=0.7     10  0.8961±0.0001  0.5527±0.0001  0.3368±0.0000  0.4618±0.0000
C alpha=0.7     15  0.9374±0.0000  0.6614±0.0001  0.3564±0.0001  0.5015±0.0001

```

Figure 4.5: Condition C ($\alpha = 0.7$): aggregated metrics (screenshot).

- Fine-tuning the CLIP vision encoder (Condition C) substantially improves retrieval compared to the frozen baselines.
- Within Condition C, $\alpha = 0.7$ performs better than $\alpha = 0.3$ across K (e.g., HR@15: 0.8925 $\rightarrow$ 0.9374; NDCG@15: 0.4169 $\rightarrow$ 0.5015).

5. Results and Trends

5.1 Aggregated Results (from Experiment Outputs)

Table 5.1 summarizes the mean metrics reported in the experiment outputs. All results are aggregated over four evaluation seeds.

Standard deviations (across seeds). We report metrics as mean $\pm$ standard deviation across four seeds. In these experiments, the “seed” affects only the evaluation ordering/shuffling (the retrieval pipeline itself is deterministic for fixed weights), and the evaluation set is large (14,218 queries), so the observed standard deviations are extremely small (typically $\approx 10^{-4}$), indicating stable estimates rather than sensitivity to randomness.

Table 5.1: Main results for Conditions A, B, and C (mean values).

Condition	K	HR@K	Recall@K	mAP@K	NDCG@K
A (vision-only)	5	0.4479	0.1608	0.1502	0.2066
A (vision-only)	10	0.5152	0.1973	0.1411	0.2075
A (vision-only)	15	0.5530	0.2211	0.1407	0.2129
B ($\alpha = 0.3$)	5	0.3376	0.1211	0.0964	0.1384
B ($\alpha = 0.3$)	10	0.4229	0.1598	0.0951	0.1474
B ($\alpha = 0.3$)	15	0.4617	0.1797	0.0958	0.1532
B ($\alpha = 0.7$)	5	0.4107	0.1527	0.1220	0.1734
B ($\alpha = 0.7$)	10	0.5142	0.2041	0.1216	0.1870
B ($\alpha = 0.7$)	15	0.5655	0.2335	0.1238	0.1968
C ($\alpha = 0.3$)	5	0.6817	0.3219	0.2465	0.3306
C ($\alpha = 0.3$)	10	0.8292	0.4682	0.2651	0.3800
C ($\alpha = 0.3$)	15	0.8925	0.5721	0.2812	0.4169
C ($\alpha = 0.7$)	5	0.7812	0.3967	0.3162	0.4108
C ($\alpha = 0.7$)	10	0.8961	0.5527	0.3368	0.4618
C ($\alpha = 0.7$)	15	0.9374	0.6614	0.3564	0.5015

5.2 Within-Ablation Comparisons

Effect of K . Within each condition, HR@K and Recall@K increase with K (more chances to include at least one relevant item in the returned set).

Seed-to-seed stability. The reported standard deviations across the four seeds are extremely small (typically $\approx 10^{-4}$). This indicates that the evaluation is stable and that differences between configurations (e.g., $\alpha = 0.3$ vs $\alpha = 0.7$) are not driven by seed noise.

Effect of α within fusion-based conditions

For conditions with caption fusion (B and C), increasing α increases the weight on the visual embedding and decreases reliance on caption/text embeddings. In both conditions, $\alpha = 0.7$ consistently outperforms $\alpha = 0.3$ across all K .

Figure 5.1 isolates the effect of changing α within each fusion-based condition at $K = 10$.

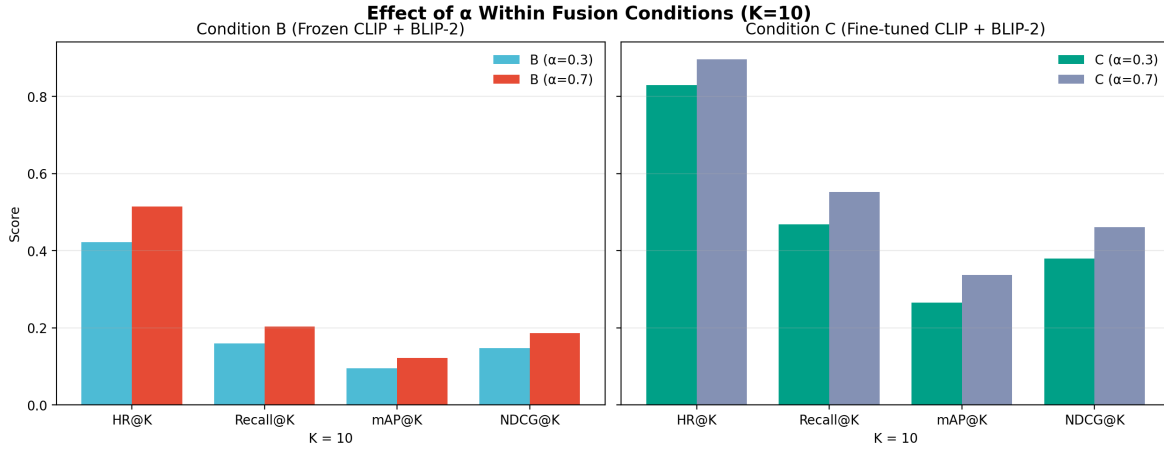


Figure 5.1: Effect of α within Conditions B and C at $K = 10$ (generated by `plot_results.py`).

Condition B details (mean $\pm$ std over 4 seeds)

Table 5.2 reports the Condition B metrics with standard deviations.

Table 5.2: Condition B aggregated over 4 seeds (mean $\pm$ std).

Config	K	HR@K	Recall@K	mAP@K	NDCG@K
B $\alpha = 0.3$	5	0.3376 $\pm$ 0.0002	0.1211 $\pm$ 0.0000	0.0964 $\pm$ 0.0001	0.1384 $\pm$ 0.0001
B $\alpha = 0.3$	10	0.4229 $\pm$ 0.0001	0.1598 $\pm$ 0.0001	0.0951 $\pm$ 0.0001	0.1474 $\pm$ 0.0001
B $\alpha = 0.3$	15	0.4617 $\pm$ 0.0002	0.1797 $\pm$ 0.0001	0.0958 $\pm$ 0.0001	0.1532 $\pm$ 0.0001
B $\alpha = 0.7$	5	0.4107 $\pm$ 0.0002	0.1527 $\pm$ 0.0001	0.1220 $\pm$ 0.0001	0.1734 $\pm$ 0.0001
B $\alpha = 0.7$	10	0.5142 $\pm$ 0.0001	0.2041 $\pm$ 0.0001	0.1216 $\pm$ 0.0001	0.1870 $\pm$ 0.0001
B $\alpha = 0.7$	15	0.5655 $\pm$ 0.0001	0.2335 $\pm$ 0.0001	0.1238 $\pm$ 0.0001	0.1968 $\pm$ 0.0001

5.3 Across-Ablation Comparisons

Figure 5.2 provides an at-a-glance comparison of all metrics at $K = 10$ across configurations.

Using Table 5.1:

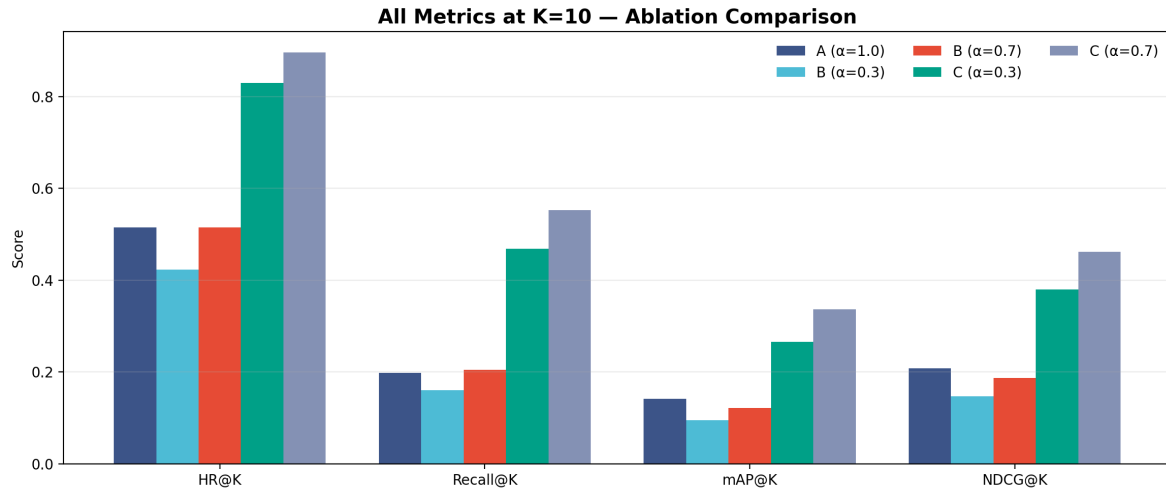


Figure 5.2: All metrics at $K = 10$ across A/B/C configurations (generated by `plot_results.py`).

- **A vs B (caption fusion without fine-tuning):** The impact of fusion depends strongly on α . With $\alpha = 0.3$ (more text weight), Condition B underperforms Condition A, suggesting that noisy/ambiguous captions can dilute discriminative visual cues. With $\alpha = 0.7$ (more vision weight), the text signal acts as a *supporting* cue and Condition B improves HR@K/Recall@K over A (e.g., HR@15: 0.5530 $\rightarrow$ 0.5655; Recall@15: 0.2211 $\rightarrow$ 0.2335), while ranking-sensitive metrics (mAP/NDCG) remain comparable.
- **A/B vs C (fine-tuning + fusion):** Condition C provides large gains across all metrics and K , indicating that adapting CLIP to the dataset distribution is the dominant contributor to retrieval quality.
- **Best overall configuration:** Condition C with $\alpha = 0.7$ yields the strongest HR@K, Recall@K, mAP@K, and NDCG@K among the reported configurations.

6. Streamlit Application

We implemented an interactive Streamlit application (`vr_final_project_app/streamlit_app.py`) to demonstrate the end-to-end flow:

6.1 User Flow

1. **Upload:** the user uploads a JPG/PNG query image. The app normalizes it to RGB and stores it in session state (keyed by a file hash) so changing the upload resets the downstream state.
2. **Detect & crop:** a YOLO detector runs on the image at a user-controlled confidence threshold. The UI shows one crop per predicted class (best box per class), with padding applied; the user selects which crop to use. To avoid duplicate/overlapping crops across categories (e.g., topwear vs fullbody), we apply **cross-class non-maximum suppression (NMS)**: after selecting the best box per class, boxes are compared across classes and lower-confidence boxes are suppressed when IoU exceeds a fixed threshold. If YOLO weights are unavailable, the app falls back to a mock center-crop.
3. **Category:** the app lets the user select a coarse clothing category (Top wear / Bottom wear / Full body). If YOLO returns a class label (e.g., `clothes_type_1`), the category is auto-filled, but the user can override it.
4. **Confirm gate:** retrieval is blocked until the user clicks *Confirm & Search*. This prevents indexing/search from running on accidental crops.
5. **Index build (first run):** on the first confirmed search, the app loads gallery records (from `catalog.json` if available, otherwise by scanning the gallery folder) and builds a vector index. Gallery embeddings are cached to disk (`artifacts/cache/`) to avoid rebuilding in subsequent runs.
6. **Search & rerank:** the selected crop is embedded with CLIP (fine-tuned weights if present, otherwise base CLIP), retrieved with HNSW (or brute-force fallback), and optionally re-ranked using BLIP Image-Text Matching (ITM) between the query image and each candidate's stored caption.
7. **Display:** the app renders a grid of top- K results showing the candidate image, item id, CLIP similarity score, optional ITM score, category, and caption.

6.2 Application Screenshots

Figure 6.1 shows the Streamlit landing page with pipeline status badges and the step-based flow. Figure 6.2 shows the upload and detection preview where the user selects a YOLO crop and category before triggering retrieval. Figure 6.3 shows the final top- K retrieval grid with CLIP similarity, optional ITM scores, and the retrieved item ids.

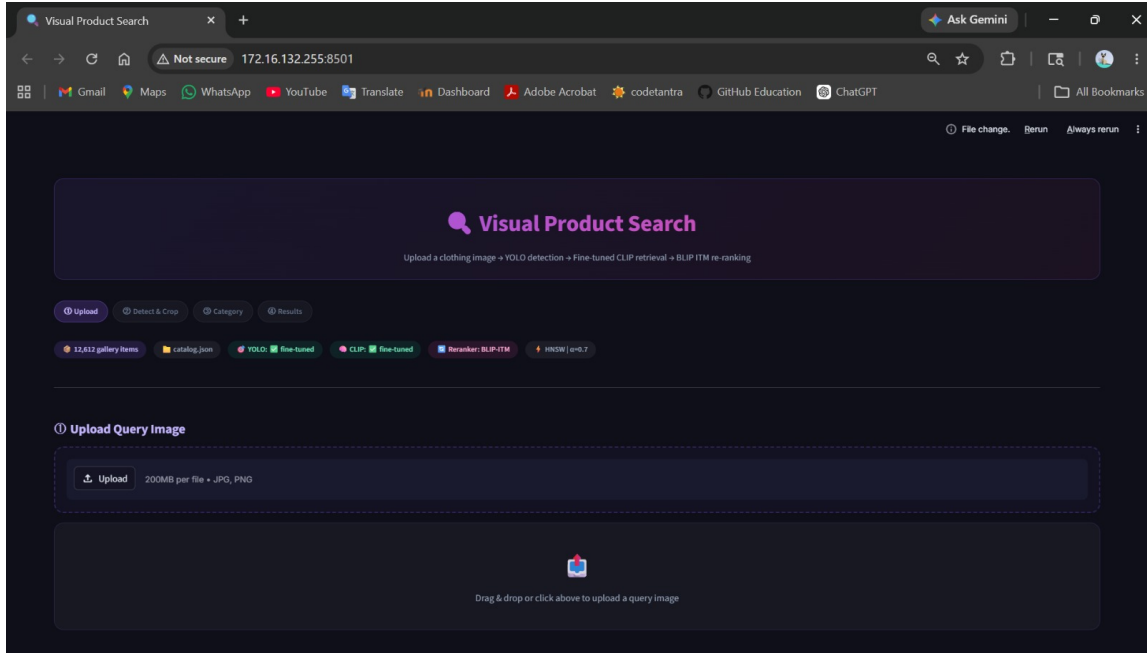


Figure 6.1: Streamlit app landing page.

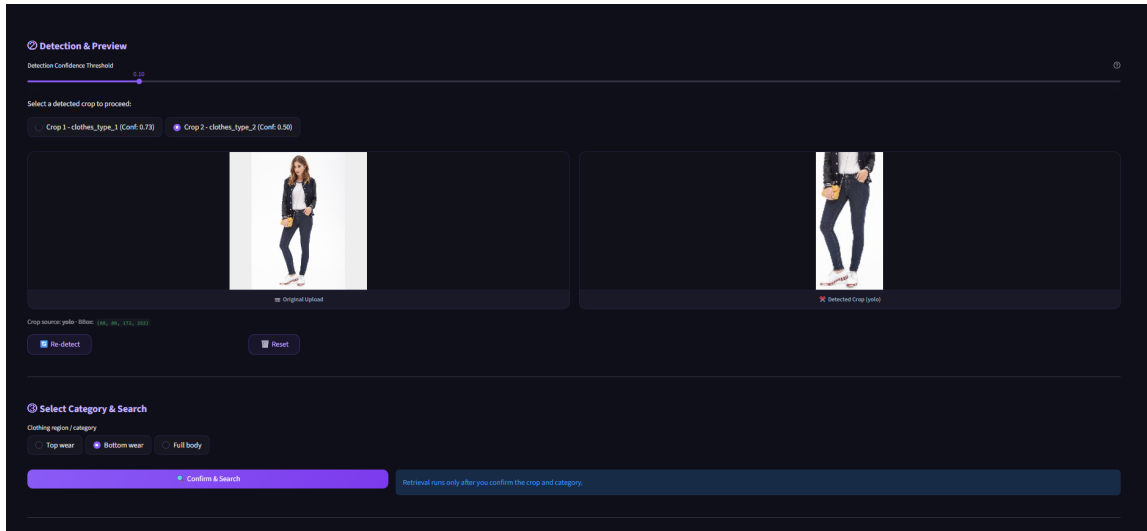


Figure 6.2: Upload and detection preview (original image vs. selected crop).

The confirmation step is enforced as a gate: retrieval does not start until the crop is approved.

Figure 6.4 shows an example output produced by our offline inference script on a test query: the query image is highlighted, and the remaining tiles show the top retrieved gallery images for that query.

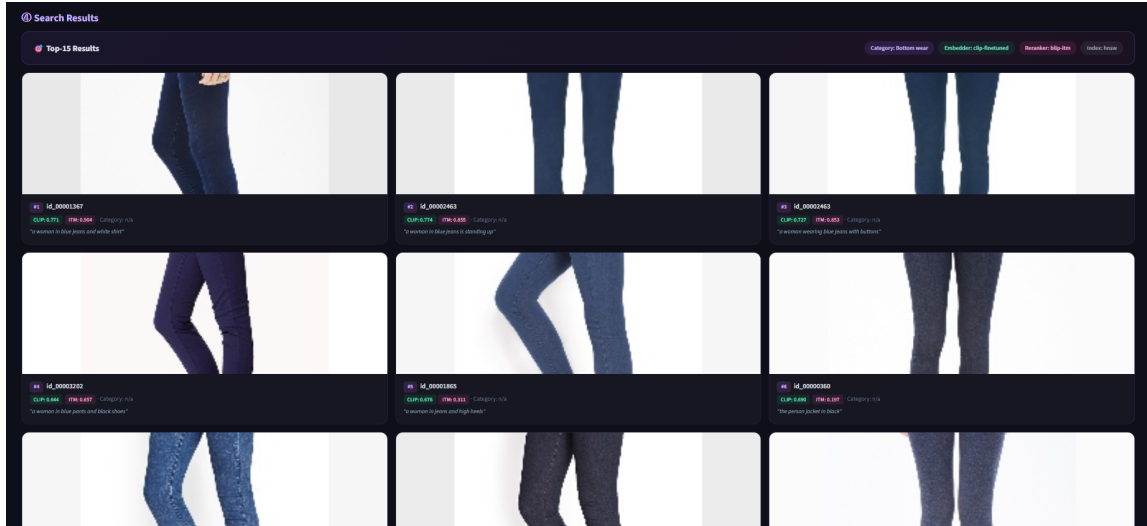


Figure 6.3: Search results view (top- K retrieved items with scores and captions).

6.3 Software Architecture

To keep the application modular and maintainable, the codebase wraps each ML component behind a small adapter interface:

- **Detector:** `YoloDetector.detect_and_crop` returns one best crop per class above threshold, with padding; `MockDetector` provides a safe fallback.
- **Embedder:** `ClipFinetunedEmbedder` (loads `clip_best.pt`) or `ClipEmbedder`. Both support image embeddings; the fine-tuned embedder also supports batched embedding for fast index build.
- **Index:** `HnswIndex` (cosine similarity) with `BruteForceIndex` fallback.
- **Reranker:** `BlipItmReranker` scores query image vs. candidate captions and reorders candidates; `NoOpReranker` is used when dependencies are unavailable or mock mode is enabled.
- **RetrievalPipeline:** builds the gallery index using fused *gallery* embeddings $\alpha \mathbf{v} + (1 - \alpha) \mathbf{t}$ (with on-disk caching) and executes query-time search + reranking.

The retrieval backend supports either **HNSW** (`hnswlib`) or a **brute-force** numpy fallback depending on the environment.

6.4 Configuration and Execution

The app is configured via environment variables (e.g., data root, index backend, model weights). The repository README documents expected data layout and the following commands:

- Install dependencies: `pip install -r requirements.txt`
- Run: `streamlit run streamlit_app.py`

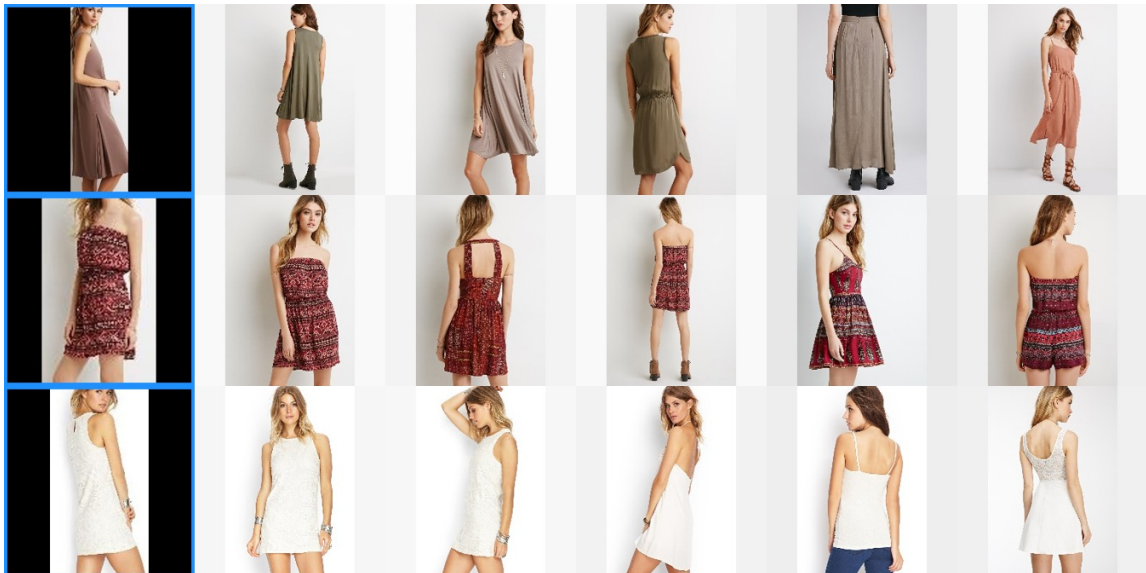


Figure 6.4: Offline inference example on a test image: highlighted query and top retrieved results.

7. Key Findings and Takeaways

- **Fine-tuning matters most:** Condition C consistently outperforms the frozen baselines, indicating that adapting CLIP to the dataset distribution is the dominant contributor to retrieval gains.
- **Fusion is helpful when weighted correctly:** Within fusion-based conditions, $\alpha = 0.7$ performs better than $\alpha = 0.3$, suggesting that captions add useful context but should not overwhelm visual cues.
- **Condition B is α -sensitive:** With $\alpha = 0.3$ (text-heavy), fusion can reduce performance relative to the vision-only baseline due to caption noise/ambiguity. With $\alpha = 0.7$ (vision-heavy), text acts as a supporting cue and yields small HR/Recall improvements over Condition A, while mAP/NDCG remain comparable.
- **Results are stable:** Very small standard deviations across seeds ($\approx 10^{-4}$) indicate that the evaluation is consistent and the observed differences are not driven by seed noise.
- **App flow mirrors the pipeline:** The Streamlit demo enforces a crop-and-category confirmation gate, then runs indexed retrieval and optional BLIP-ITM reranking to present top- K results.

8. Conclusion

This project delivers a complete visual product search pipeline with:

- YOLO-assisted cropping to reduce background noise,
- CLIP-based embeddings for robust retrieval,
- caption fusion and fine-tuning (Condition C) that yields major metric improvements over the baseline,
- a Streamlit demo application with a user-confirmation gate and modular, swappable components.

Future work could include stronger fine-tuning objectives (e.g., cross-modal contrastive learning), better caption prompting, and persistent on-disk indexing for faster app startup.